Unit IV:
API Integration and Data Management

Session 5

Royal University of Bhutan

Outline

- Handling API requests and responses
  - Fetch API and Axios library
  - Async/await and Promise-based requests
  - Request interceptors and headers
  - Response parsing and data transformation
  - Cancelling requests and timeout handling
- File upload implementations - Multipart form data

Handling API requests and responses - **Fetch API**

- The Fetch API is a built-in JavaScript API for making HTTP requests.
- It provides a more powerful and flexible feature set than older techniques like XMLHttpRequest.

Fetch_API_vs_Axios

Handling API requests and responses - **Fetch API**

```
// Basic GET request
fetch('https://api.example.com/data')
  .then(response => {
    // Check if the response is successful
    if (!response.ok) {
      throw new Error(`HTTP error! Status: ${response.status}`);
    }
    return response.json(); // Parse the JSON response
  })
  .then(data => {
    console.log('Data received:', data);
    // Process the data
  })
  .catch(error => {
    console.error('Fetch error:', error);
  });
```

Handling API requests and responses - **Fetch API**

```
// POST request with JSON data
fetch('https://api.example.com/users', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
  },
  body: JSON.stringify({
    name: 'Tashi',
    email: 'tashi@example.com',
  }),
})
  .then(response => response.json())
  .then(data => console.log('User created:', data))
  .catch(error => console.error('Error creating user:', error));
```

Handling API requests and responses - **Axios Library**

- Axios is a third-party HTTP client library that simplifies making HTTP requests.
- It offers more features (mostly built-in) than the native Fetch API.
- Automatically transforms response data (JSON parsing is built-in).
  - Fetch API doesn't automatically handle JSON conversion (*response.json()* is needed)

Fetch_API_vs_Axios
https://axios-http.com/

Handling API requests and responses - **Axios Library**

```
// Basic GET request
axios.get('https://api.example.com/data')
  .then(response => {
    console.log('Data received:', response.data);
    // Process the data
  })
  .catch(error => {
    console.error('Error:', error);
    if (error.response) {
      // The server responded with a status code outside the 2xx range
      console.error('Response data:', error.response.data);
      console.error('Response status:', error.response.status);
    }
  });
```

Handling API requests and responses - **Axios Library**

```
// POST request with JSON data
axios.post('https://api.example.com/users', {
  name: 'Tashi',
  email: 'tashi@example.com',
})
  .then(response => console.log('User created:', response.data))
  .catch(error => console.error('Error creating user:', error));
```

Async/await and Promise-based Requests

Feature	Promise-based	Async/await
Syntax and readability	Uses `.then()` and `.catch()` chains	Uses more synchronous-looking code with `async` functions and `await` keywords
Error handling	Relies on `.catch()` blocks or second argument to `.then()`	Uses standard `try/catch` blocks
Control flow	Chain-based flow can become complex (promise hell)	Linear code flow

Promises_and Async/await

Promise-based Requests example

```
fetch('https://example.com/posts/1')
  .then(response => response.json()) // Convert response to JSON
  .then(data => console.log(data)) // Handle data
  .catch(error => console.error('Error:', error)); // Handle errors
```

Promises_and Async/await

Async/await Requests example

```
async function fetchData() {
  try {
    const response = await fetch('https://example.com/posts/1');
    const data = await response.json(); // Convert response to JSON
    console.log(data);
  } catch (error) {
    console.error('Error:', error); // Handle errors
  }
}

fetchData();
```

Promises and Async/await

Request Interceptors

- Request interceptors allow you to **modify or examine HTTP requests** before they are sent to the server.
- They are particularly useful for:
  - Adding authentication tokens to every request
  - Logging request details
  - Transforming request data
  - Adding common headers

https://blog.jobins.jp/axios-interceptors-with-practical-examples

Response Parsing

JSON Parsing

(with Fetch API)

```
fetch('https://api.example.com/products')
  .then(response => response.json()) // Parse JSON
  .then(data => {
    console.log('Parsed data:', data);
  });
```

JSON Parsing

(with Axios)

```
// Axios automatically parses JSON responses
axios.get('https://api.example.com/products')
  .then(response => {
    console.log('Parsed data:', response.data);
  });
```

Data Transformation

The response data often needs modification to match the structure required by your application.

Common use cases:

- Date and Time Formatting
- String Manipulation
- Flattening Nested Data
- Unit Conversion …etc

```
// Convert array of objects to a different structure
const originalData = [
  { id: 1, name: 'John', role: 'Developer' },
  { id: 2, name: 'Sarah', role: 'Designer' }
];

// Map to a new structure
const transformed = originalData.map(item => ({
  userId: item.id,
  displayName: item.name,
  jobTitle: item.role
}));
```

Cancelling Requests and Timeout Handling

Request Timeouts

Response timeout refers to the duration for which a client will wait for a response from the server before considering the request as failed.

Cancelling Requests

When you need to manually cancel requests, such as when a user navigates away from a page or performs an action that makes the pending request irrelevant.

Response Timeout

Timeout Handling

```
// Set timeout for a single request
axios.get('https://api.example.com/data', {
  timeout: 5000, // 5 seconds
})
  .then(response => console.log(response.data))
  .catch(error => {
    if (axios.isCancel(error)) {
      console.log('Request timed out');
    } else {
      console.error('Error:', error);
    }
  });

// Or set a default timeout for all requests
const api = axios.create({
  baseURL: 'https://api.example.com',
  timeout: 10000, // 10 seconds
});
```

Cancelling Requests

```
// Create a cancel token source
const CancelToken = axios.CancelToken;
const source = CancelToken.source();

// Make a request with the cancel token
axios.get('https://api.example.com/data', {
  cancelToken: source.token
})
  .then(response => console.log(response.data))
  .catch(error => {
    if (axios.isCancel(error)) {
      console.log('Request cancelled:', error.message);
    } else {
      console.error('Error:', error);
    }
  });

// Cancel the request
source.cancel('Operation cancelled by the user');
```

File upload - Multipart form data

Multipart form data is a specific encoding type (`multipart/form-data`) used in HTML forms, particularly for uploading files.

How Multipart Form Data Works

When a form with `enctype="multipart/form-data"` is submitted, the browser:

1. Creates a unique boundary string to separate different form fields
2. Formats each form field with headers and content
3. Sends all data in a single HTTP request

Multipart Form Data

Basic HTML Form for File Uploads

```html
<form id="upload-form" action="/upload" method="post" enctype="multipart/form-data">
  <div>
    <label for="username">Username:</label>
    <input type="text" id="username" name="username">
  </div>
  <div>
    <label for="file-input">Select a file:</label>
    <input type="file" id="file-input" name="file">
  </div>
  <button type="submit">Upload</button>
</form>
```

References

Fetch_API_vs_Axios

https://axios-http.com/

Promises_and Async/await

Interceptors

Response Timeout

Multipart Form Data